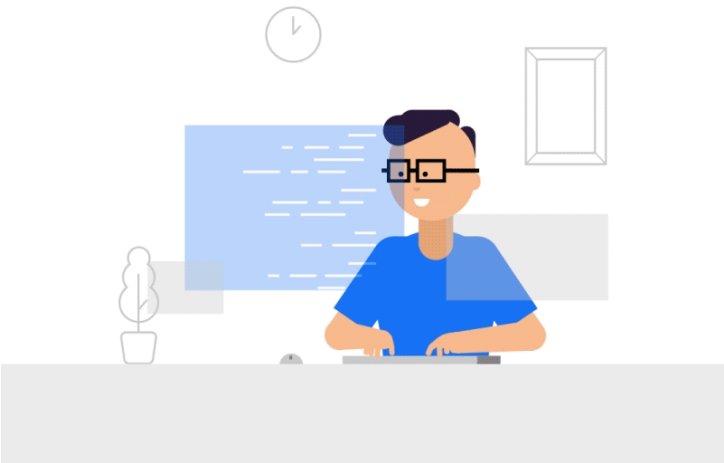
05 Web scraping

Scripting

Elke Boonen, Dries Geysens & Tristan Vandevelde

Learning objectives (ECTS)



- Learn the **syntax of Python**
- **Explore possibilities** regarding working with **data** (file, web API, web scraping...)
- Create **scripts**
- **Automate** processes

Learning materials

- **Canvas** LMS https://thomasmore.instructure.com/
  - Presentations & Assignments (CodeGrade)
  - E-Book: Automate the boring stuff with Python (https://automatetheboringstuff.com)
- **Online** https://www.python.org/ and https://pypi.org/ (search packages)
- **Github** Scripting (code from slides) & Scripting-Students (code from class)
- **Software**
  - https://www.python.org/downloads/ (IDLE Python)
  - https://www.anaconda.com/
  - Love to work with Visual Studio Code? Install Python extensions!

Schedule

Before Easter break	**After Easter break**
01 Python basics	08 Scheduling
02 Python basics	09 Images
03 Python basics	10 Databases
04 Strings & regex	11 Packaging
05 Files (IO)	12 Conclusion/exam prep
06 API	*13 Extra recap*
07 Webscraping	

Schedule is always subject to unexpected circumstances

Evaluation

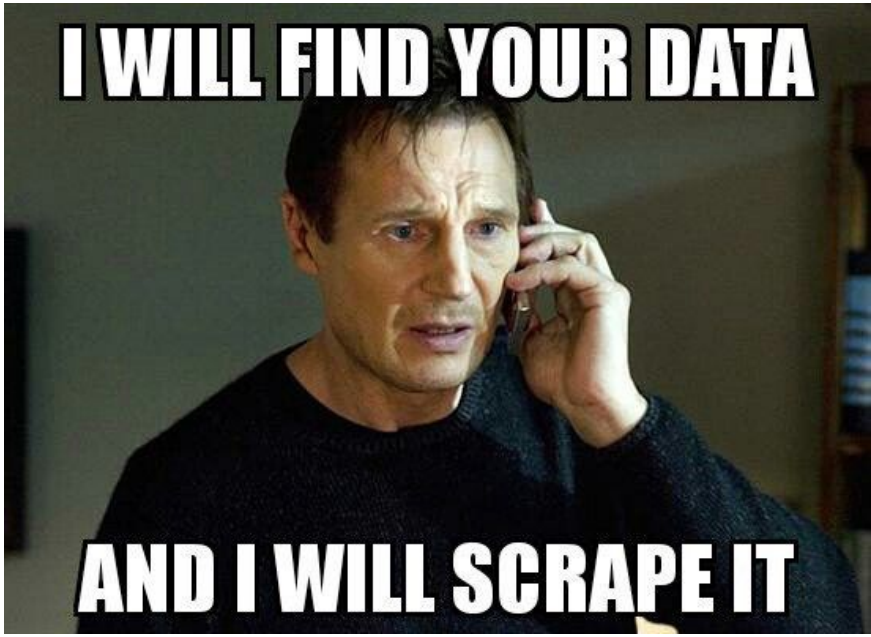
- **1st term**
  - Permanent Evaluation (30 %):
    - CodeGrade exercises (each week)
  - Take-home exam (70 %)
- **2nd term**
  - Computer Exam (100 %)

05 Webscraping

- What is web scraping?
- Scrape the beautiful soup

What is web scraping?

- **Not all data is always accessible in APIs** ... but maybe it is readable somewhere on a web page? Web scraping ensures that we still can get that data!
- Web scraping **extracts underlying HTML code**, so it is important to know your HTML-tags!

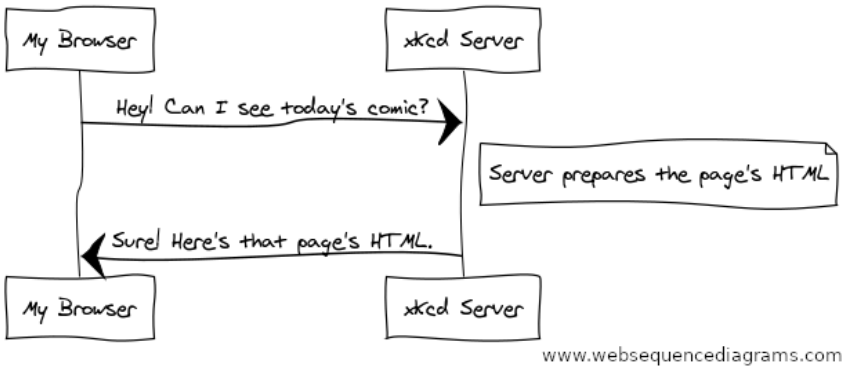


Is web scraping legal?

- Web scraping itself is **not illegal**,
  but you have to do it in an **ethical way (**some guidelines)
  - **Don't scrape** other people's sites **without permission**. Respect the rules of every site (Terms of Service, read robots.txt etc...)
  - Do **not** request data from the website too aggressively (**spamming**) as this may break the website. One request for one webpage per second is good practice.
  - **Don't publish any content** you find that was not intended to be published
- The layout of a website may change, so make sure to revisit the site and rewrite your code as needed

Another request

- To open a webpage: **send HTPP request (url) and get response back (HTML)**
- Work with package **requests**
  - send out a get-request
  - get text back
  - text = content of webpage




```python
import requests
url = "https://en.wikipedia.org/wiki/Soup"
r = requests.get(url)
text = r.text      #returns html as a string
print(text)
```

It's messy in the soup

- Received data is = page source without code formatting 😨
  = combination of HTML, CSS & JS in simple plain text!
- Without any web fundamentals knowledge = hours of decluttering and working with regex, stringparsing etc...

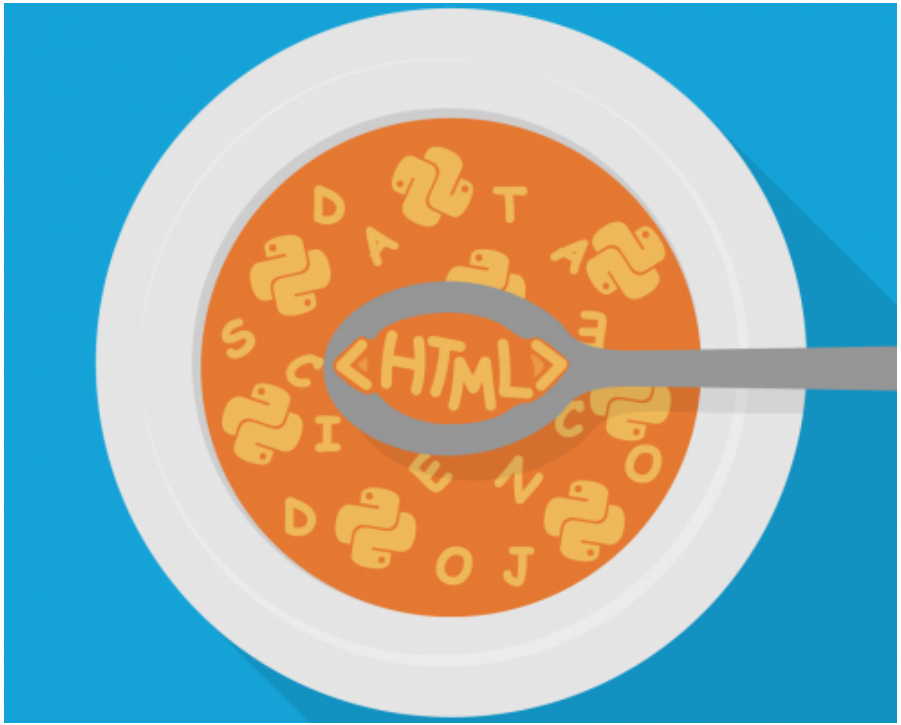
```
tle="Soep (voedsel) - Dutch" lang="nl" hreflang="nl" class="
age-link interwiki-nds-nl"><a href="https://nds-nl.wikipedia
s-NL" class="interlanguage-link-target">Nedersaksies</a></li
kipedia.org/wiki/%E3%82%B9%E3%83%BC%E3%83%97" title="スープ -
t">日本語</a></li><li class="interlanguage-link interwiki-no'
gian Bokmål" lang="nb" hreflang="nb" class="interlanguage-li
wiki-nn"><a href="https://nn.wikipedia.org/wiki/Suppe" title
nguage-link-target">Norsk nynorsk</a></li><li class="interla
Souope" title="Souope - Norman" lang="nrf" hreflang="nrf" cl
anguage-link interwiki-oc"><a href="https://oc.wikipedia.org
nterlanguage-link-target">Occitan</a></li><li class="interla
o%CA%BBrva" title="Shoʻrva - Uzbek" lang="uz" hreflang="uz"
s="interlanguage-link interwiki-pa"><a href="https://pa.wiki
ng="pa" hreflang="pa" class="interlanguage-link-target">ਪੰਜਾਬੀ
s://pl.wikipedia.org/wiki/Zupa" title="Zupa - Polish" lang="
<li class="interlanguage-link interwiki-pt"><a href="https:/
eflang="pt" class="interlanguage-link-target">Português</a><
o.wikipedia.org/wiki/Sup%C4%83" title="Supă - Romanian" lang
i><li class="interlanguage-link interwiki-qu"><a href="https
reflang="qu" class="interlanguage-link-target">Runa Simi</a>
u.wikipedia.org/wiki/%D0%A1%D1%83%D0%BF" title="Суп - Russia
</a></li><li class="interlanguage-link interwiki-sco"><a hre
="sco" hreflang="sco" class="interlanguage-link-target">Scot
```

05 Webscraping

- ~~What is web scraping?~~
- Scrape the beautiful soup

Turn in into something beautiful

- Let's install another package!
- Use Beautifulsoup
  - **pip install beautifulsoup4**
- Beautiful Soup is a Python library for getting data out of HTML, XML, and other markup languages...



Turn in into something beautiful

- Import **Beautifulsoup**

```
tle="Soep (voedsel) - Dutch" lang="nl" hreflang="nl" class="
age-link interwiki-nds-nl"><a href="https://nds-nl.wikipedia
s-NL" class="interlanguage-link-target">Nedersaksies</a></li
kipedia.org/wiki/%E3%82%B9%E3%83%BC%E3%83%97" title="スープ -
t">日本語</a></li><li class="interlanguage-link interwiki-no'
gian Bokmål" lang="nb" hreflang="nb" class="interlanguage-li
wiki-nn"><a href="https://nn.wikipedia.org/wiki/Suppe" title
nguage-link-target">Norsk nynorsk</a></li><li class="interla
Souope" title="Souope - Norman" lang="nrf" hreflang="nrf" cl
anguage-link interwiki-oc"><a href="https://oc.wikipedia.org
nterlanguage-link-target">Occitan</a></li><li class="interla
o%CA%BBrva" title="Sho'rva - Uzbek" lang="uz" hreflang="uz"
s="interlanguage-link interwiki-pa"><a href="https://pa.wiki
ng="pa" hreflang="pa" class="interlanguage-link-target">ਪੰਜਾ
s://pl.wikipedia.org/wiki/Zupa" title="Zupa - Polish" lang="
<li class="interlanguage-link interwiki-pt"><a href="https:/
eflang="pt" class="interlanguage-link-target">Português</a><
o.wikipedia.org/wiki/Sup%C4%83" title="Supă - Romanian" lang
i><li class="interlanguage-link interwiki-qu"><a href="https
reflang="qu" class="interlanguage-link-target">Runa Simi</a>
u.wikipedia.org/wiki/%D0%A1%D1%83%D0%BF" title="Cyn - Russia
</a></li><li class="interlanguage-link interwiki-sco"><a hre
="sco" hreflang="sco" class="interlanguage-link-target">Scot
```

```python
from bs4 import BeautifulSoup
import requests

url = 'https://en.wikipedia.org/wiki/Soup'
r = requests.get(url)
soup = BeautifulSoup(r.text)
print(soup.prettify())
```

```
href="https://jv.wikipec
      Jawa
     </a>
    </li>
    <li class="interlar
     <a class="interlar
"kn" lang="kn" title="ಸೂ
      ಕನ್ನಡ
     </a>
    </li>
```

Beautiful minestrone soup!

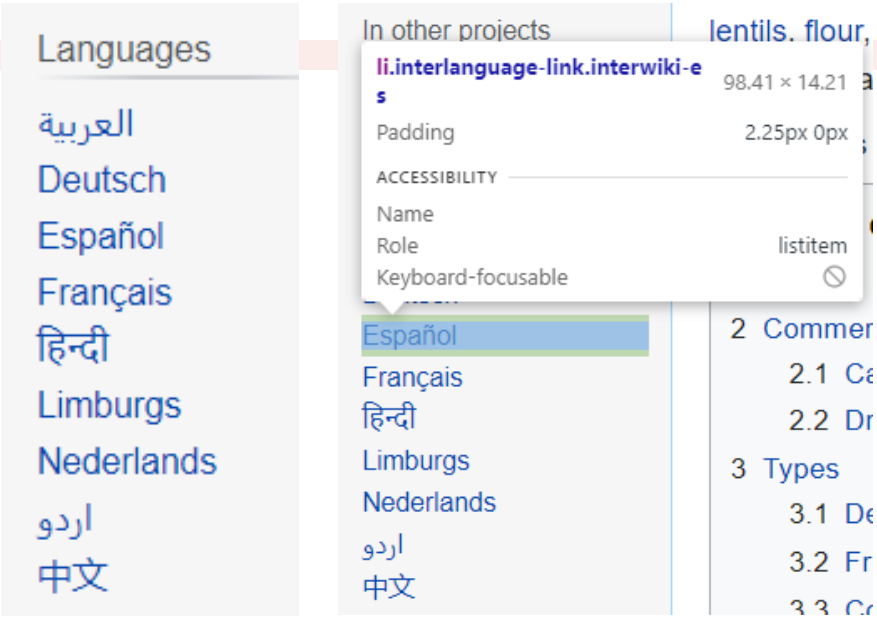
- Easy access to all kinds of information, check the documentation of package
  - https://www.crummy.com/software/BeautifulSoup/bs4/doc/
- **Method get_text** is useful in only showing the text parts of an entire document or a tag! Once you have text, well... then you can start playing with string functions.

```python
from bs4 import BeautifulSoup
import requests

url = 'https://en.wikipedia.org/wiki/Soup'
r = requests.get(url)
soup = BeautifulSoup(r.text)

print(soup.title)
print(soup.title.get_text())
print(len(soup.get_text()))
print(soup.get_text()[0:500].split())
print(soup.get_text().count("soup"))
```

```
<title>Soup - Wikipedia</title>
Soup - Wikipedia
25436
['Soup', '-', 'Wikipedia', 'Soup', 'From', 'Wiki
'search', 'For', 'other', 'uses,', 'see', 'Soup'
r', 'verification.', 'Please', 'help', 'improve'
'Unsourced', 'material', 'may', 'be', 'challenge
'·', 'books', '·', 'scholar', '·', 'JSTOR', '(Fe
te', 'message)', 'Soup']
210
```

Inspect elements!

- **Before** you actually start **scraping**,
  it is extremely important that you study the page source yourself (HTML is no case for serendipity!)
- **Investigate** and **inspect elements**
  - open developers toolbox in Chrome (F12)
  - use CTRL+SHIFT+C on an element




```
▶<h3 id="p-lang-label" class="vector-menu-heading">…</h3>
▼<div class="vector-menu-content">
  ▼<ul class="vector-menu-content-list">
    ▼<li class="interlanguage-link interwiki-af" style="dis
    y: none;">
        <a href="https://af.wikipedia.org/wiki/Sop" title="S
        – Afrikaans" lang="af" hreflang="af" class="interlan
        ge-link-target">Afrikaans</a>
      </li>
    ▶<li class="interlanguage-link interwiki-ak" style="dis
    y: none;">…</li>
```

The real thing!

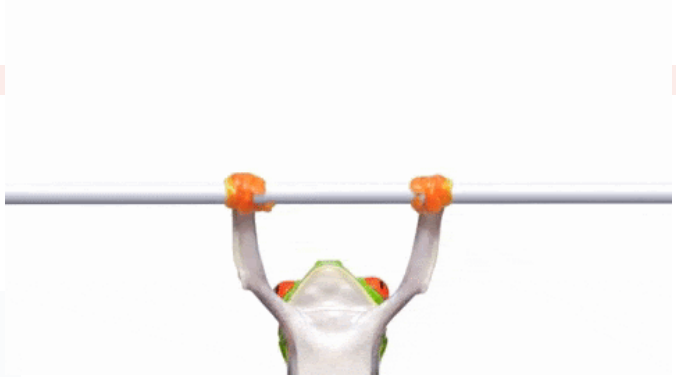
- We can use the methods **find** and **find_all**
  - find: return first element
  - find_all: return all found elements

```python
from bs4 import BeautifulSoup
import requests

url = 'https://en.wikipedia.org/wiki/Soup'
r = requests.get(url)
soup = BeautifulSoup(r.text)
a_tags = soup.find_all('a',{ "class" : "interlanguage-link-target" })

for a in a_tags:
    print(a.get('title'))

#or more pythonic ;)
names = [a['title'] for a in tags]
print(names)
```

```
Sop - Afrikaans
Nkwan - Akan
Suppe - Alemannisch
حساء - Arabic
Sopa - Asturian
Şorba - Azerbaijani
স্যুপ - Bangla
Thng - Chinese (Min Nan)
Һурпа - Bashkir
Суп - Belarusian
Супа - Bulgarian
Supa - Bosnian
Soubenn - Breton
Sopa - Catalan
```

Don't forget the notebook!

- Check the notebook with extra exercises!



ElkeBoonen / **Scripting**

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

master | 1 branch | 0 tags | Go to file | Add file

Elke Boonen We interrupt this program to annoy you and make things generally more... ··· 3f8e2

.vscode	Black Knight: "Tis but a scratch." King Arthur: "A scratch? Your
01 Python basics	Black Knight: "Tis but a scratch." King Arthur: "A scratch? Your
02 Strings & regex	"One day, lad, all this will be yours." "What, the curtains?"
03 Files	I'm not dead yet... I feel happy!
04 API	Nobody expects the spanish inquisition
05 Webscraping	We interrupt this program to annoy you and make things gen

Practice makes perfect

Scrape a bit more on https://en.wikipedia.org/wiki/Soup and

- actually download those images
- print all the Traditional regional varieties
- print all books for further reading
- get more information about every found book on https://openlibrary.org/dev/docs/api/books

Challenge yourself on https://scrapethissite.com/

- find all flags from all countries
- find all islands (in the name)
- find the smallest & largest country through scraping
- find all countries with an A...
- ...

It seems that it would be useful to store the information from https://scrapethissite.com/ in a database t
you agree?

]:
```
from bs4 import BeautifulSoup
import requests
import re

#GO GO GO
```

Practice makes perfect!

- *Check out the extra (not graded) exercises in the notebooks!* *See Github*
- Do your exercises, spend the hours!

Say what? How many hours?

6 SP = 6 * 28 hours = 168 hours

Lessons = 12 * 3 hours = 36 hours

Take-home exam = 16 hours

Exercise = 168-36-10 = 116 hours